

MATCH SCORING EXPLAINED (PLAIN ENGLISH)

Source file: src/utils/matchScoring.js

This document explains two different outputs the app can compute for a hike or event:

- Match score: a straight average of how close each preference is (0% to 100%).
 - Cosine similarity: a content-based score using a feature vector and angle math.
- They often move together, but they are not the same thing. The match score is a simple average. Cosine similarity looks at the overall shape of the user vs event vectors.

1) WHAT DATA IS USED

User preferences: experience level, preferred difficulty, preferred duration, budget range, preferred trail type, preferred distance, preferred elevation.

Event details: difficulty (or distance/elevation/duration if difficulty is missing), duration, price, trail type (and text description), distance, elevation.

2) HOW EACH PERCENT IS CALCULATED

Each parameter becomes a percent from 0.00 to 1.00 (0% to 100%).

- Difficulty (experience + preferred difficulty): both labels are mapped to an index in the order Beginner, Intermediate, Technical, Expert. If the gap is 0, percent = 1. If the gap is 1, percent = $1 - 1/3 = 0.6667$. If the gap is 2, percent = 0.3333 . If the gap is 3, percent = 0.
- Duration / Distance / Elevation: percent = $1 - |actual - preferred| / \max(actual, preferred)$. Exact match gives 1.00. Farther away lowers the percent.
- Budget: the text range is parsed into numbers. The cap is max if present, else min, else midpoint. If price is at or below the cap, percent = 1. If price is above, percent = cap / price.
- Trail type: if the user has a preferred trail type, the event gets 1 if it matches the event's trailType or appears in its description; otherwise 0. If the user did not set a preference, this value is null and is not averaged.

3) FINAL MATCH SCORE

All non-null percents are averaged with equal weight. Missing user preferences (null values) are ignored rather than treated as 0.

4) COSINE SIMILARITY (CONTENT-BASED)

Cosine similarity uses a numeric vector for the user and the event. Each number is normalized to 0..1. Numeric values are scaled by caps: duration/12, distance/40, elevation/2000, price/12000. Difficulty labels are mapped to scores (beginner=0.2, intermediate=0.6, technical=1, expert=1).

User vector order:

- [experienceScore, preferredDifficultyScore, durationScore, budgetScore, trailPrefFlag, distanceScore, elevationScore]

Event vector order:

- [difficultyScore, difficultyScore, durationScore, priceScore, trailScore, distanceScore, elevationScore]

Formula: $\text{cosine} = \text{dot}(\text{user}, \text{event}) / (||\text{user}|| * ||\text{event}||)$, clamped to 0..1.

Because this compares the direction of the vectors, cosine similarity can stay fairly high even if one parameter is low, as long as the overall shape is similar.

5) WORKED EXAMPLE (STEP BY STEP)

User preferences:

- Experience = Intermediate, Preferred difficulty = Intermediate, Duration = 6 hrs, Budget = PHP 2,000 - 3,000, Trail type = Forest, Distance = 12 km, Elevation = 800 m.

Event details:

- Difficulty = Technical, Duration = 8 hrs, Price = PHP 3,500, Trail type = Rocky, Distance = 10 km, Elevation = 1,200 m.

Per-parameter percents:

- Experience vs difficulty: gap = 1 (Intermediate -> Technical), percent = $1 - 1/3 = 0.667$ (66.7%).

- Preferred difficulty vs difficulty: same calculation = 0.667 (66.7%).

- Duration: $1 - |8 - 6| / 8 = 0.750$ (75.0%).

- Budget: cap = 3000; price = 3500 -> $3000/3500 = 0.001$ (0.1%).

- Trail type: Forest vs Rocky -> 0.000 (0.0%).

- Distance: $1 - |10 - 12| / 12 = 0.833$ (83.3%).

- Elevation: $1 - |1200 - 800| / 1200 = 0.667$ (66.7%).

Match score = average of the seven percents = 0.512 (51.2%).

Cosine similarity values:

- User vector = [0.600, 0.600, 0.500, 0.208, 1.000, 0.300, 0.400].

- Event vector = [1.000, 1.000, 0.667, 0.292, 0.000, 0.250, 0.600].

- Dot product = 1.848; $||\text{user}|| = 1.490$; $||\text{event}|| = 1.718$.

- Cosine similarity = 0.722 (72.2%).

6) REFERENCES (CODE LOCATIONS)

All references are in src/utils/matchScoring.js:

- Difficulty mapping and order: lines 1-11.

- Normalization caps: lines 13-17.

- Numeric percent formula: lines 42-58.

- Difficulty matching percent: lines 132-147.

- Difficulty inference from metrics: lines 150-197.
- Budget parsing: lines 209-246.
- Budget percent formula: lines 248-281.
- Normalization helpers (duration/distance/elevation/price): lines 283-309.
- User vector: lines 333-362.
- Event vector: lines 365-396.
- Collect per-parameter percents: lines 399-449.
- Match score average: lines 28-40 and 453-459.
- Cosine similarity: lines 462-489.
- Breakdown filtering (min share): lines 18 and 500-503.